EECS 348 Assignment 5 — UML Use Case Diagram

System: Sudoku Solver System (Assignment5.cpp) | **Revised by:** Justin Whitaker | **Revision date:** October 29, 2026
Original diagram: Assignment 4 Use Case Diagram received from Aidan Wiehl. **GenAI used to modify it:** Claude Opus 5.5 (Anthropic), through the Claude Code desktop app on my computer.

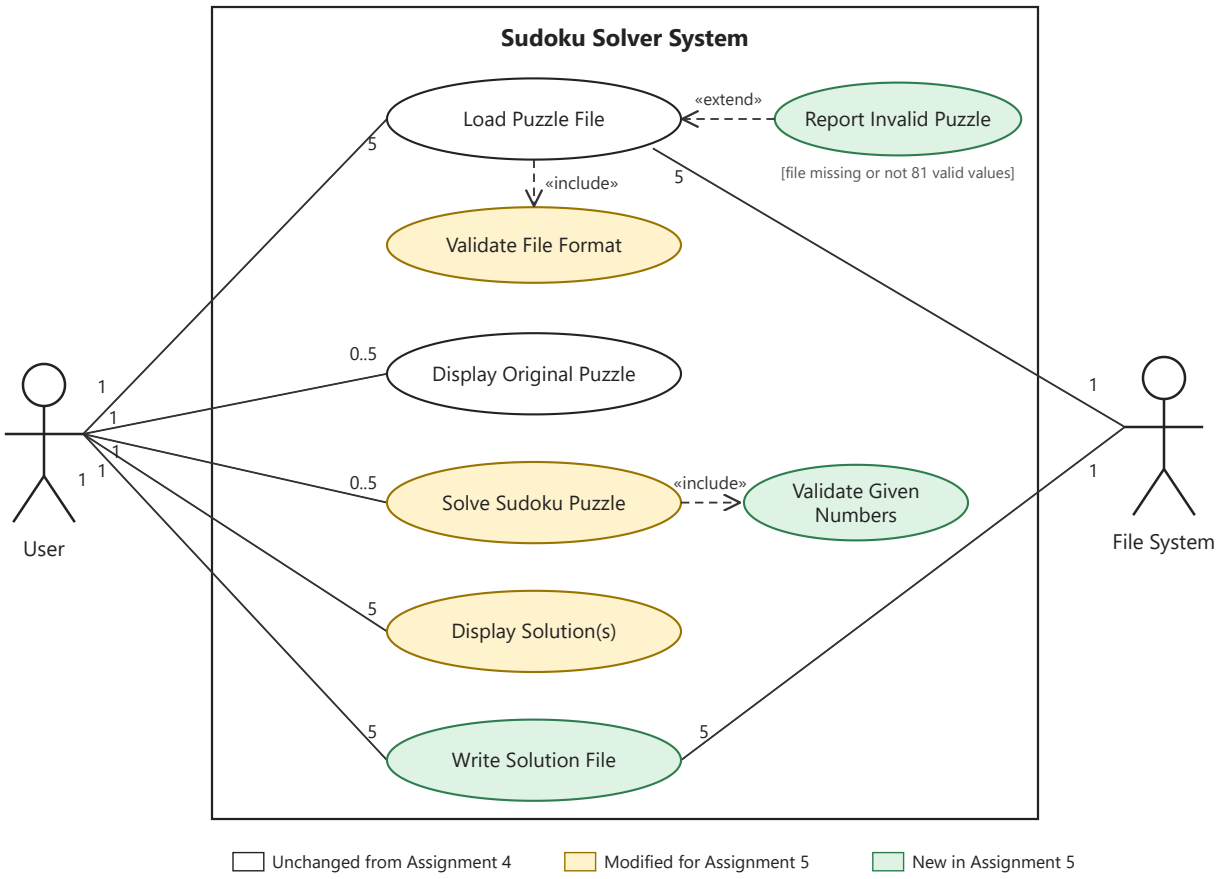


Figure 1. Modified Use Case Diagram for the Assignment 5 Sudoku Solver System. Multiplicities: the number next to an actor is how many of that actor take part in one use case; the number next to a use case is how many times that use case happens in one program run (5 puzzle files).

What changed from the received Assignment 4 diagram: (1) added the *Write Solution File* use case, connected to both the User and the File System, because the program now saves every puzzle's results to `solutionN.txt`; (2) added *Validate Given Numbers* as an «include» of *Solve Sudoku Puzzle*; (3) added *Report Invalid Puzzle* as an «extend» of *Load Puzzle File* for missing or badly formatted files; (4) updated *Validate File Format*, *Solve Sudoku Puzzle*, and *Display Solution(s)* to match the Assignment 5 program; (5) replaced the "1 to 1" multiplicities with ones that show five puzzle files per run.

a. Identify / Discover the Use Cases

#	Use case	Status	Actors and relationships
1	Load Puzzle File	Unchanged	User (primary), File System (secondary); «include» Validate File Format
2	Validate File Format	Modified	Included by Load Puzzle File

3	Report Invalid Puzzle	New	«extend» Load Puzzle File when the file is missing or not 81 valid values
4	Display Original Puzzle	Unchanged	User
5	Solve Sudoku Puzzle	Modified	User; «include» Validate Given Numbers
6	Validate Given Numbers	New	Included by Solve Sudoku Puzzle
7	Display Solution(s)	Modified	User
8	**Write Solution File**	New	User (primary, receives the file), File System (secondary, stores it). This is the use case added by Assignment 5 and is fully described in sections c and d.

b. Brief Descriptions

1. **Load Puzzle File:** This use case allows the User to have the system read one puzzle file (puzzle1.txt – puzzle5.txt) from the File System into memory.
2. **Validate File Format:** A required part of loading. The system checks that the file holds exactly 81 values and that each value is a digit 1–9 or an underscore (an empty cell). In Assignment 4 this check was shown on the diagram but not done by the code; it is now enforced.
3. **Report Invalid Puzzle:** Optional behavior that runs only when the puzzle file cannot be opened or fails the format check. The system tells the User what is wrong and treats the puzzle as having no solution.
4. **Display Original Puzzle:** The system shows the User the puzzle file name and the unsolved 9x9 grid exactly as it was read.
5. **Solve Sudoku Puzzle:** This use case allows the User to have the system find every solution of a loaded puzzle with a recursive depth-first search with backtracking. Each solution is handed to Display Solution(s) and Write Solution File as soon as it is found, instead of being stored.
6. **Validate Given Numbers:** A required part of solving. Before searching, the system checks that the given digits do not already repeat in a row, column, or 3x3 block; if they do, the puzzle has no solution.
7. **Display Solution(s):** The system shows the User every solution, numbered, followed by "Solutions found: n", or the message "No solution found."
8. **Write Solution File:** This use case allows the User to receive the results for each puzzle in a file named `solutionN.txt` (N is the puzzle number), stored by the File System. The file holds every solution in the same format as the puzzle files followed by "Solutions found: n", or the single line "No solution found."

c. Step-by-Step Flow of Events (Outline)

Write Solution File

Basic flow

1. User runs the program.
2. System names the output file for the puzzle (puzzleN.txt → solutionN.txt).
3. System asks the File System to create the solution file.
4. System loads and validates the puzzle (Load Puzzle File).
5. System solves the puzzle (Solve Sudoku Puzzle).
6. System writes each solution to the file as it is found.

7. System writes the final line "Solutions found: n".
8. System saves (closes) the file.
9. System tells the User the file name it wrote.
10. Steps 2–9 repeat for the next puzzle file until all five are done.

Alternative flows

- **A1. Puzzle file missing or badly formatted** (step 4): Report Invalid Puzzle runs; the file gets the single line "No solution found."
- **A2. Given numbers clash** (step 5): the System reports it; the file gets "No solution found."
- **A3. No solution exists** (step 5): the file gets the single line "No solution found."
- **A4. Solution file cannot be created** (step 3): the System shows an error; solving and displaying still happen.

Whole program run (updated from the received diagram)

1. User runs the program.
2. For each of the five puzzle files, the System shows the puzzle file name.
3. System creates the matching solution file in the File System.
4. System reads the puzzle from the File System and validates its format («include»); if it is invalid, Report Invalid Puzzle runs («extend»).
5. System displays the original puzzle.
6. System validates the given numbers («include») and runs the recursive backtracking search.
7. System displays and writes each solution as it is found.
8. System displays and writes "Solutions found: n" or "No solution found."

d. Detailed Description

Use Case: Write Solution File

Primary actor	User (the person who runs the program and reads the solution files)
Secondary actor	File System (stores puzzle1.txt – puzzle5.txt and the solution files)
Goal	As a User, I want each puzzle's solutions saved to solutionN.txt so that I can check or submit the results without copying them from the screen.
Preconditions	The program is in the same folder as puzzle1.txt – puzzle5.txt, and the folder can be written to.
Postconditions	solution1.txt – solution5.txt exist. Each holds every solution in the puzzle-file format followed by "Solutions found: n", or exactly the line "No solution found."

1.1 Basic Flow

1. User Starts the Program

The use case starts when the User runs `./Assignment5.exe`.

The System builds its list of the five puzzle files and starts with puzzle1.txt.

2. System Names and Creates the Solution File

The System replaces the "puzzle" part of the file name with "solution" (puzzle3.txt becomes solution3.txt).

The System asks the File System to create the file, replacing any old copy. The file is created before the puzzle is read, so every puzzle always gets a solution file.

3. System Loads and Validates the Puzzle

The System shows the banner with the puzzle file name, then reads the file from the File System (Load Puzzle File).

Validate File Format confirms there are exactly 81 values and each is 1–9 or "_". The System then shows the original puzzle to the User.

4. System Checks the Given Numbers

Validate Given Numbers confirms that no given digit repeats in its row, column, or 3x3 block.

5. System Writes Each Solution as It Is Found

The recursive search (Solve Sudoku Puzzle) fills the empty cells. Every time the board becomes full, the System adds 1 to the solution count, shows the solution to the User as "Solution k:", and writes the same 9 rows to the solution file.

Each row is written in the same format as the puzzle files: each value followed by a space. A blank line follows each solution. The search then backtracks and continues until every possibility has been tried.

6. System Writes the Summary Line

When the search ends, the System writes "Solutions found: n" as the last line of the file and shows the same line to the User.

7. System Saves the File and Confirms

The System closes the file so the File System saves it, then shows "Solutions written to: solutionN.txt".

Steps 2–7 repeat for puzzle2.txt through puzzle5.txt. The use case ends after solution5.txt is saved.

1.2 Alternative Flows

A1. Puzzle File Missing or Badly Formatted (at step 3)

If the file cannot be opened, the System shows "Error opening file: puzzleN.txt". If it has too few or too many values, or a character other than 1–9 or "_", the System shows "Invalid puzzle format in file: puzzleN.txt" (Report Invalid Puzzle).

The System skips steps 3–5 for this puzzle, writes the single line "No solution found." to the solution file, and continues at step 7.

A2. Given Numbers Break the Rules (at step 4)

The System shows "The given numbers break the Sudoku rules." and skips the search. It writes "No solution found." and continues at step 7.

A3. No Solution Exists (at step 6)

If the search finishes with a count of 0 (puzzle4.txt is an example), the System shows and writes the single line "No solution found." instead of "Solutions found: n".

A4. Solution File Cannot Be Created (at step 2)

The System shows "Error creating output file: solutionN.txt". It still loads, solves, and displays the puzzle for the User, but does not show the "Solutions written to" confirmation.

Use Case: Solve Sudoku Puzzle (updated detailed description)

1. User Initiates Puzzle Solving

Running the program starts the solving for each loaded puzzle. The System creates a solver object linked to that puzzle's solution file, with a solution count of 0.

2. System Validates the Given Numbers («include»)

The System checks every given digit against its row, column, and 3x3 block. If any digit repeats, the search is skipped (see A2 above).

3. System Scans for an Empty Cell

The System looks row by row for the next cell containing "_". If none is left, the board is a complete solution, and the System displays and writes it immediately (Write Solution File, step 5), then returns to look for more.

4. System Tests Candidate Digits

For the empty cell, the System tries the digits 1 to 9 in order. It keeps only the digits that are not already in the cell's row, column, or 3x3 block.

5. System Recurses and Backtracks

For each allowed digit, the System places it and calls the search again for the rest of the board. When that call returns, the System resets the cell to "_" (backtracking) and tries the next digit. Every digit is tried, so ALL solutions are found, not just the first.

6. System Presents the Result

After the search ends, the System shows and writes "Solutions found: n", or "No solution found." when the count is 0.

Multiplicities used in Figure 1

Association	Multiplicity	Reason

User – any use case (actor end)	1	One person runs the program; no use case involves more than one User.
User – Load Puzzle File	5	Every run loads (or tries to load) exactly five puzzle files.
User – Display Original Puzzle	0..5	A puzzle that is missing or invalid is not displayed, so 0 to 5 puzzles are shown.
User – Solve Sudoku Puzzle	0..5	Only puzzles that load and have valid given numbers are searched.
User – Display Solution(s), Write Solution File	5	Every puzzle always ends with a result line on the screen and a solution file, even if it is "No solution found."
File System – Load Puzzle File, Write Solution File	1 / 5	One file system serves the run; it supplies five puzzle files and stores five solution files.

Source note: this document modifies the Assignment 4 Use Case Diagram received from Aidan Wiehl. The GenAI used for the modification was Claude Opus 5.5 (Anthropic) through the Claude Code desktop app; I reviewed the result against the Assignment 5 program.